
Chapter 5 : Array and Pointers

Composite Type

A data type is called **simple** if variables of that type can store only one value at a time.

Composite type or aggregate type is types that we can use to define collections of data such that we can manipulate an entire collection as a unit, but can still refer to individual components of a given datum by name.

Composite Types are derived from more than one primitive type.

Examples

Array, struct, unions

A ***data structure*** is a particular way of storing and organizing composite data in a computer memory so that it can be used efficiently

Array

Array is an elementary data structure that exists as built-in in most programming languages.

Array is a **collection structure** composed of a **fixed number of elements** wherein all of the elements have **the same data type**.

The elements are stored in **contiguous computer memory** so that it allows **random access** of its elements, i.e. elements can be accessed using indexing by address calculation.

Arrays are suitable for representing composite data which consist of many similar, individual items. Examples include: a list of names, a table of world cities and their current temperatures, or the monthly transactions for a bank account.

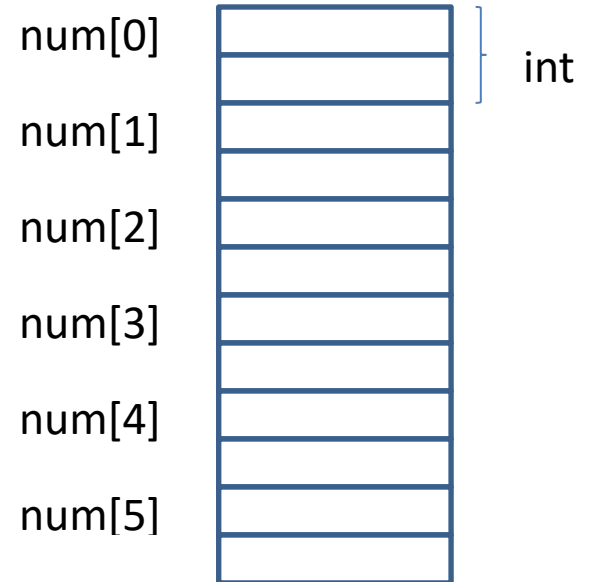
Array

In general, only the array itself has a symbolic name, not its elements.

Each element is identified by **an index** which denotes the position of the element in the array.

The number of elements in an array is called its **dimension**. The dimension of an array **is fixed** and predetermined; it cannot be changed during program execution.

(see side figure) An array of **6** integers of type **int** as a collection referred by a name **num** and individual elements referred by subscript as num[0], num[1],...



Array Categorization

Array can be categorized into two part

1) One dimensional Array:

collection of a fixed number of elements (of the same type) arranged in one dimension as a list

2) Multi dimensional :

collection of a fixed number of components (of the same type) arranged in multi dimensions

One-dimensional Array:

Declaration of One-dimensional Array

An array must be declared before it is used. **Definition and declaration** tell the compiler **the name, type, and dimension** of the array.

The general syntax for one-dimensional array is:

type variable_name[dimension/size];

The dimension of the array must be **an integer constant or integral constant expression** and must have a value **at compilation time**.

Example

```
float score[20]; //score can store 20 float value
```

```
char letterGrade[50]; //letterGrade can store 50 char value
```

```
int next, score[5], max; // declare arrays and regular  
                        variables together
```

```
const unsigned int SIZE=50;
```

```
double temp[2*SIZE]; //constant expression for dimension
```

```
int i=15, temp[i]; //invalid declaration
```

Declaration...

Declaration and definition only reserve space for the elements in the array. No values will be stored. If we want to store values in the array, we must either initialize the elements, read value from the keyboard, or assign value to each individual elements.

Array Initialization

Array can be initialized at the time of declaration. There are two option.

Syntax (Option 1):

type array_Name[dimension/size] = { value0, value1, ..,value2};

type array_Name[dimension/size] { value0, value1, ..,value2};

Where

- type specify that what kind of array you are declaring
- array name specify the name of the array
- dimension specifies the size of the array
- value0, value1,...,valueN specify the initial values for array_name[0], array_name[1],..., array_name[N] respectively.

Dimension is optional. The compiler can determine the dimension from the list of initial values.

If the number of initial values is less than the dimension specified, **the extra elements** of the array will be initialized to **0** for numeric type array or **null character('\0')** for character type array.

Array...

Example:

```
int num[5]={3,7,12,24,15}; //equivalent int num[]={3,7,12,24,15};  
double distance[5] = {44.14, 720.52, 96.08, 468.78, 6.28};  
double distance[5] = {0}; //all elements initialised to zéro  
double m[5] = {1.5, 2.3, 4.7}; //m[0], m[1], m[2], m[3], and m[4] are  
initialized to 1.5, 2.3, 4.7, 0.0, and 0.0 respectively.
```

Referencing Elements of an Array

An array individual elements can be accessed using an **index**. For a one-dimensional array, the first element has an index of 0, the second element has an index of 1, and so on. Array index starts from zero.

The general format for accessing an array element in one dimensional array is:

array_name[index];

Where **array_name** is the name of the array and **index** is an integral **constant** or **any expression** that evaluates to an **integral value**.

Example:

score[3] refers to the third element to the array score

Referencing...

- **An element of an array can be used any where a variable can be used.**

Assuming the following declaration

```
int arr[20]={1}, i=1, y=3;
```

An element of array can be assigned:

```
arr[4] = 55; //The fifth element of the array arr assigned 55
```

It can be used in an expression(as lvalue and rvalue):

```
arr[2*i +1] = 3*y + arr[2*i]; //the 4th element is assigned 10
```

It can be used in an input-output statement:

```
cin >> arr[18]; //the 19th element will be an integer read from  
keyboard
```

```
cout << arr[15]; //The 16th will be displayed on the screen
```

Referencing...

Consider the following declaration for the next proposition:

```
int lista[]={5, 7, 9, 6, 4}, listb[5];
```

- Attempting to access a **nonexistent array element** leads to a serious **runtime error** (called ‘**index out of bounds**’ error). The compiler won’t complain if you assign a value to the nonexistent element. Your code will compile.

```
listb[5]=lista[0];    // index out of bounds error
```

```
cout << lista[-1] ;   // index out of bounds error
```

- C++ does not allow **aggregate operations** on an array:

```
listb=lista;    //illegal. Copy element by element
```

```
cin >> listb;   //illegal! Read element by element
```

```
cout << listb;   //illegal! Display element by element
```

```
Copy element by element
```

Array Processing

Some basic operations performed on a one-dimensional array are:

- Initialization
- Filling array from keyboard
- Outputting data stored in an array
- Finding the largest and/or smallest element
- Each operation requires ability to step through the elements of the array
- Easily accomplished by a loop

Initialization

A loop is very useful for initialization of an array specially if the values of the array follows some pattern:

Example:

```
int odd[50]
for(int i = 0; i < 50; i++)
    odd[i] = 2*i + 1;
```

Filling array from Keyboard

A loop is very useful to obtain the values of the array from the keyboard:

Example:

```
float score[60]
for(int i = 0; i < 60; i++){
    cout << "score[" << i << "] = ";
    cin << score[i];
}
```

Outputting Data Stored in an Array

A loop is very useful to display the values of the array :

Example:

```
float score[60];
```

```
for(int i = 0; i < 60; i++)
```

```
    cout << "score[" << i << "] = " << score[i]<<endl;
```

Finding The Smallest, Largest, Range and Average Value

Write a program that collects the score of students and determine the smallest, largest, range and average of the scores

Range Based For Loop and Array

A range based style loop is designed **to cycle through a collection of objects**, such as an array, vector in strictly sequential fashion, from start to finish.

The general form of the for-each version of the for is shown here:

```
for(range_decleration : collection){  
    statements  
}
```

There are two types of ranged based loops

1. **Normal Iterators:** an ordinary temporary variable is declared as the iterator, and the iterator gets a copy of the current loop item **by value**. Any changes made to the temporary copy will not get reflected in the original iterable.

Syntax

```
for (datatype iterator : collection) {  
    // operation are performed here  
}
```

Range Based For Loop...

```
int arr[]={12, 20, 50,54, 72, 101};  
for(int x:arr){  
    cout<<x<<" ";  
}
```

The above code prints all the values in arr separated by space.

Range Based For Loop...

2. Reference Iterators: Reference iterators are declared as a [reference variable](#), and the iterator gets the value of the current item by reference. So the changes made inside the loop are definitely get affected in the original container itself. Syntax

```
for (datatype & iterator : collection) {  
    // operation are performed here  
}  
  
int arr[]={12, 20, 50,54, 72, 101};  
for(auto & x:arr){  
    x=2*x;  
}
```

Two-dimensional Array:

Two-dimensional Array

Two-dimensional array: collection of a fixed number of components (of the same type) arranged in two dimensions

Sometimes called **matrices or tables**

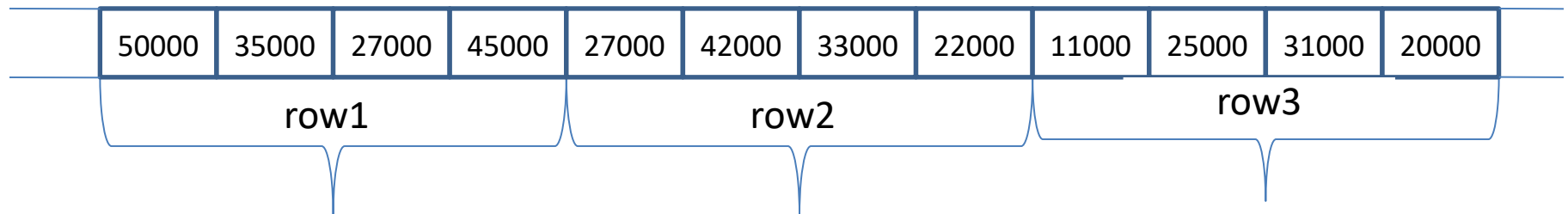
Logical View of Two Dimensional Array

The table below shows the sale amount of salesperson for the quarters of the year. This sales data can be represented using a two dimensional array, row number representing salesperson and column number representing quarter. It is a 7 by 4 two dimensional array. This is the logical view of the programmer, as seven rows of four doubles entries each.

		Quarter			
Salesperson		Q1	Q2	Q3	Q4
	Abebe	50,000.00	35,000.00	27,000.00	45,000.00
	Almaz	27,000.00	42,000.00	33,000.00	22,000.00
	Hagos	11,000.00	25,000.00	31,000.00	20,000.00
	Tesfaye	47,000.00	36,000.00	26,000.00	32,000.00
	Roman	55,000.00	45,000.00	52,000.00	50,000.00
	Eden	47,000.00	45,000.00	48,000.00	52,000.00
	Solomon	31,000.00	34,000.00	29,000.00	52,000.00

Physical View of Two-dimensional Array

The organization of the array in memory (The physical view of the programmer) is however still the same (a contiguous sequence of elements) as the one-dimensional array.



The organization of this array in memory is as 28 consecutive double elements.

Two-dimensional Array Declaration

A two dimensional array has two indices: row index and column index.

The general syntax for declaring two dimensional array is:

dataType arrayName[rowSize][columnSize];

Where dataType, arrayNname, rowSize, columnSize refer to the data type, array name, row dimension and column dimension of the array respectively.

The dimension of the array must be **an integer constant or integral constant expression** and must have a value at compilation time.

```
double sales[7][4]; //declaration for the previous sale table
```

This array consists of seven rows and four columns and is called a 7-by-4 array

Initialization of Two-dimensional Array

As with one-dimensional arrays, two-dimensional arrays can be initialized in their declaration statements by listing the initial values inside braces and separating them with commas. Additionally, braces can be used to separate rows.

Syntax:

```
dataType  arrayName[n][m] ={{value00, value01,...,value0m},  
                               {value10, value11,...,value1m},  
                               {value20, value21,...,value2m},  
                               .  
                               .  
                               {valuen0, valuen1,...,valuenm}}      .
```

- Although the commas in initialization braces are always required, the inner braces can be omitted.

Initialization...

- In two-dimensional array, if the array is completely initialized, it is possible to omit the first dimension(the row size).
- For numerical arrays, if all components of a row aren't specified, unspecified ones are set to 0

Example

```
int table[5][4]={ {0, 1, 2, 3}, {10,11,12,13}, {20, 21, 22, 23},  
                 {30, 31, 32, 33}, {40, 41, 42, 43}};
```

Or

```
int table[5][4]={0, 1, 2, 3, 10,11,12,13, 20, 21, 22, 23,  
                 30, 31, 32, 33, 40, 41, 42, 43};
```

Or

```
int table[][4]={ {0, 1, 2, 3}, {10,11,12,13}, {20, 21, 22, 23},  
                 {30, 31, 32, 33}, {40, 41, 42, 43}};
```

Referencing Array Elements for Two-dimensional Array

The individual elements of a two-dimensional array can be accessed using two indices: row and column indices.

Syntax:

array_name[rowIndex][columnIndex];

Where **array_name** is the name of the array and **rowIndex** and **columnIndex** are an integral **constant** or **any expression** that evaluates to an **integral value**.

Example:

```
double sale[7][4];
```

`sale[2][3]` refers to the element at the 3rd row and the 4th column to the array `score`

Referencing...

- **An element of a two-dimensional array can be used anywhere a variable can be used.**

Assuming the following declaration

```
int arr[2][5]={1,2,3,5,5,5,4,5,3,6}, i=1, y=3;
```

It can be assigned:

```
arr[1][4] = 55;    //The element at row index 1 and column index 4 is  
// assigned 55
```

It can be used in an expression(as lvalue and rvalue):

```
arr[0][2*i+1] = 3*y + arr[2*i][2];    // The element at row index 0 and  
//column index 3 is assigned 12
```

It can be used in an input-output statement:

```
cin >> arr[1][3];
```

```
cout << arr[0][2];
```

Processing Two-Dimensional Arrays

Processing a two-dimensional array usually requires a nested loop.

Initialization

- To initialize a particular row (i.e., fifth row) to 0
row=4;
for(i = 0; i < rowSize; i++)
 table[row][i]=0;
- To initialize the whole array to some value v
for(i = 0; i < rowSize; i++)
 for(j = 0; j < colSize; j++)
 table[i][j]=v;

Processing...

Inputting

```
for(i = 0; i < rowSize; i++)  
    for(j = 0; i < colSize; i++)  
        cin >> table[i][j];
```

Outputting

```
for(i = 0; i < rowSize; i++){  
    for(j = 0; i < colSize; i++)  
        cout << setw(5) << table[i][j] << 't';  
    cout << endl;  
}
```

Range Based For Loop For 2D array

```
int b[2][3] = { { 1, 2, 3 }, { 4, 5, 6 } };  
for (auto& outer : b) {  
    for (auto& inner : outer) {  
        std::cout << inner << std::endl;  
    }  
}
```

On the outer loop the auto and the & are required so that the array doesn't decay to a pointer.

Example

Write a program that calculates and displays the sum for each individual row, column, and the two diagonal of a square matrices of integer. The dimension of the square matrix is known to be not greater than 10. The actual dimension must be accepted from the user. The matrix is populated by accepting the data from the user.

Multi-Dimensional Array:

Multi-Dimensional Array

A multi-dimensional array can have three, four or more dimensions.

Multidimensional Array Declaration

SYNTAX

```
Type Array_Name[Dim_1][Dim_2]...[Dim_Last];
```

EXAMPLES

```
double three_d_picture[10][20][30];
```

Using Multi-Dimensional Array

A multi-dimensional array is used in a similar fashion as two-dimensional array.

Initialization

An n-dimensional array will be initialized during declaration using n level nested braces for each dimension.

Example for three-dimensional array

```
int arr[2][3][4] = {{{1,2,3,4}, {5,6,7,8},{9,10,11,12}},  
{{13,14,15,16}, {17,18,19,20},{21,22,23,24}}}
```

It is possible to omit size of first dimension but not other dimensions

Note: A three-dimensional array is simply an array of two-dimensional array.

Using Multi-Dimensional Array

Referencing

An n-dimensional array will be referenced using n indices for each dimension. Example

```
arr[2][3][1]=55;
```

Processing

An n-dimensional array will be processed using n level nested loops.

Limitation of an Array

The dimension of an array **is fixed** and predetermined at compile time; it cannot be changed during program execution. It can't be resized. For this reason the programmer is forced to estimate the size of the array which may result in **overestimation** and hence waste of memory space or **underestimation** in which case the program is unable to hold all the information required and ultimately crash .

Pointer

Pointer-Concept

The memory of your computer can be imagined as a succession of memory cells, each one of the minimal size that computers manage (one byte).

These single-byte memory cells are numbered in consecutive way. This number is called **the address of that byte**. **Hexadecimal number is used to represent these addresses.**

000A1051	6
000A1052	
000A1053	
000A1054	2
000A1055	
000A1056	
000A1057	7
000A1058	
000A1059	
000A1060	1
000A1061	0
000A1062	
000A1063	8
000A1064	
000A1065	

Pointer and Variable

A variable is implemented as a sequence of adjacent memory locations. For example

`float var1=27.34;`

`char var2='A'; short var3;`

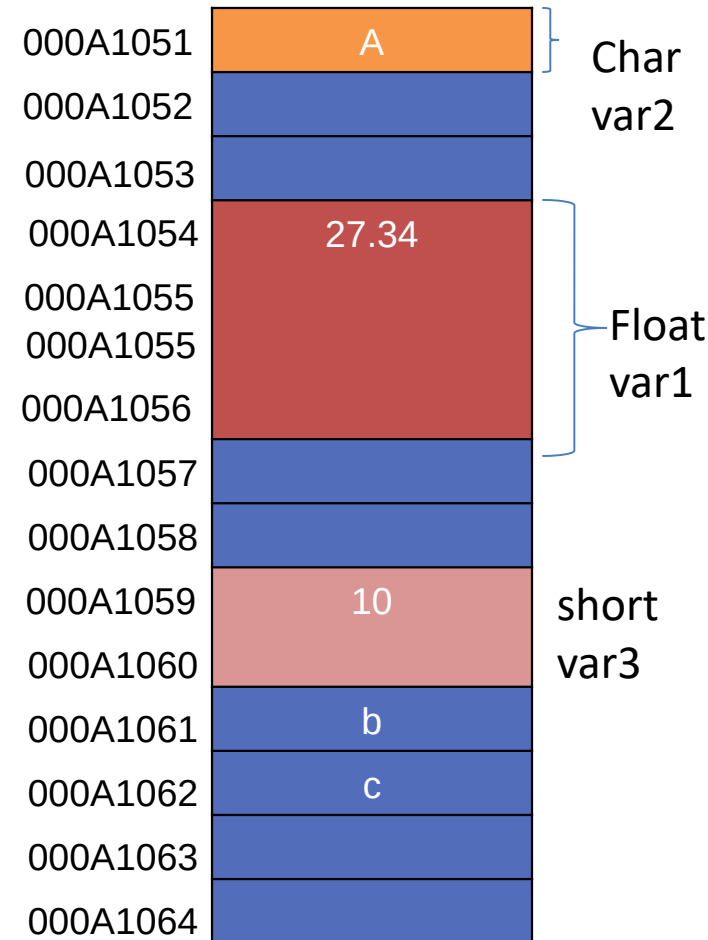
`var1` is implemented as 4 bytes of consecutive memory.

`var2` is implemented as 1 bytes of memory.

`var3` is implemented as 2 bytes of consecutive memory

The memory **address of the first byte** can be used as names for the variables.

Thus a **pointer to the memory address of the first byte of the variable** provides an indirect way of accessing variable.



Pointer Type

A pointer data type is a derived data type whose value may be any of the **memory address** available in the computer.

A pointer data type is a data type for containing **an address rather than a data value**.

It is integral, similar to int.

The size depends on the number of bytes in which the target computer stores a memory address(usually 4 bytes)

Pointer Value

C++ allows you to obtain **the memory address of the first byte of a variable, i.e. the pointer to the variable, through the address operator(&).**

The address operator(&) provides a pointer constant to any named location in memory.

Syntax:

&var_name

Where var_name is a valid variable name.

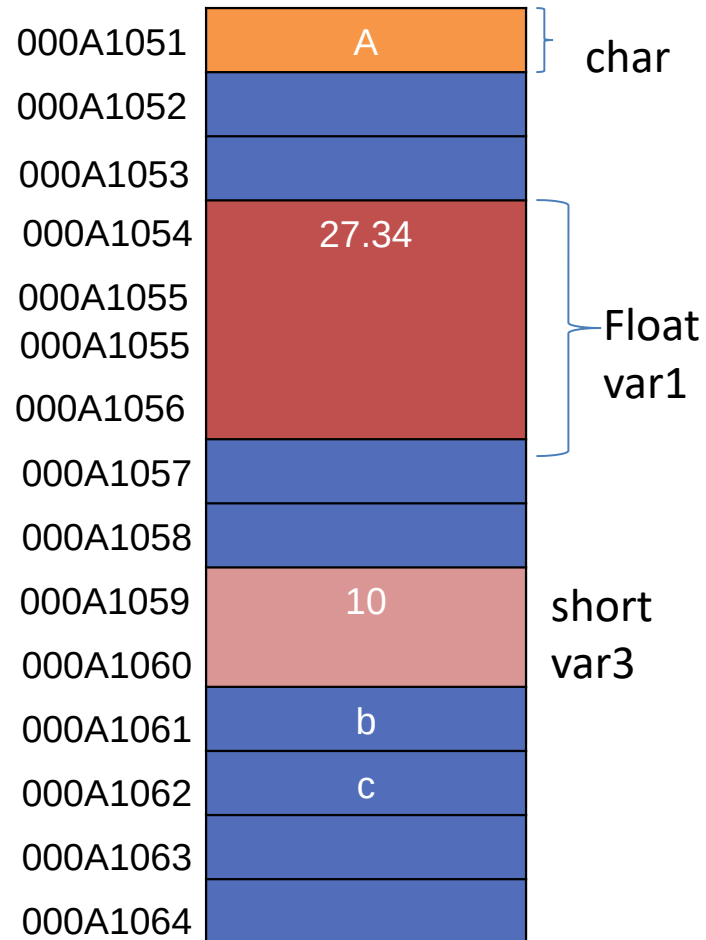
Example: consider the previous slide and what will be printed

```
float var1=27.34;
```

```
char var2='A';
```

```
cout << "Address of var1: " << &var1 << '\n';
```

```
cout << "Address of var2: " << &var2 << '\n';
```



Pointer Variable

Even though a pointer value is a memory address and a memory address is a number, you cannot store a pointer in a variable of type int or double.

A pointer variable is a variable that allows to store and manipulate a **pointer** values(address of a variable).

Declaring a Pointer Variable

Like any other variable a pointer variable should be declared prior of use.

Different types uses different numbers of bytes and different internal formats for storing values. Therefore, a pointer declaration must specify what type of data to which the pointer points.

Declaration syntax:

`dataType * ptr1, * ptr2, ..., * ptrLast;`

The variable ptr1, ptr2,...ptrLast can hold pointers to variables of type dataType only but it cannot normally contain a pointer to a variable of some other type.

Example:

`double * ptrSale; //ptrSale is a pointer variable that can point a
double variable`

`int *p1, *p2, v1, v2; //p1, p2 are pointers, v1,v2 are int`

Pointer Initialization

You can use **the address operator &** to determine the address of a variable, and you can then assign that address to **a pointer variable**.

A pointer can be initialized during declaration by assigning it the address of an existing variable. Then it is said to be **it points to that variable**.

SYNTAX

```
dataType var1, var2,..., varLast;
```

```
dataType *pvar1=&var1, *pvar2=&var2, ..., *pvarLast=&varLast;
```

EXAMPLE

```
float data = 50.8;
```

```
float *ptr = &data; //ptr points to data
```



A pointer can be initialized to a NULL(0) during declaration . Then it is said to be it points to nothing.

```
int *ip = 0; // points to nothing
```

```
float *fp = NULL; // points to nothing
```

Pointer Initia...

Pointers can also be initialized or set a value after declaration.

Example:

```
int ival1;
```

```
float fval1;
```

```
int * iptr1, *iptr2;
```

```
float *fptr1,*fptr2 ;
```

```
fptr1=&fval1;//fptr1 points to fval1
```

```
fptr2=fptr1;
```

```
fptr2=0; //fptr1 points to nothing
```

```
iptr1=&ival1; //iptr points to ival1
```

```
iptr2=&fval1; //invalid assignment!!
```



Note: For a type T, T* is the type “**pointer to T**”. That is, a **variable of type T*** can hold the address of an object **of type T**.

Accessing Variables Through Pointers

Pointers stores a reference to another variable. Pointers are said to "**point to**" the variable whose reference they store.

Using a pointer we can indirectly access **the value** stored in the variable which it points to. C++ provides **the indirection (or dereference) operator (*)** for doing this.

Syntax:

***ptr_variable**

Where ptr_variable is a pointer variable.

Accessing...

EXAMPLE

```
double x=0; *ptr1, *ptr2;
```

```
ptr1=&x;
```

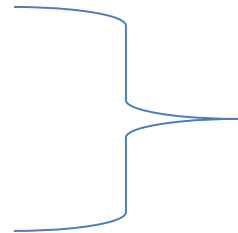
```
ptr2=ptr1;
```

```
cout << x;
```

```
cout << *ptr1;
```

```
cout << *ptr2;
```

```
*ptr1=6;
```



The three statements are equivalent

```
cout << x << " " << *ptr2; //What is the output?
```

Note:

- It is an error to dereference a pointer whose value is NULL. It is recommended first to check for NULL before dereferencing
- your program may also crash with run time error if you try to dereference uninitialized pointer variable.

Pointer Operation

- Assignment – the value of one pointer variable can be assigned to another pointer variable of the **same type**. Example:

```
int *ptr1, *ptr2, ival=0;
```

```
float *ptr3, fval=5;
```

```
ptr1=&val; //valid?
```

```
ptr1=&fval; // valid?
```

```
ptr3=ptr1; // valid?
```

```
ptr2=&ptr1; // valid?
```

```
ptr3=fval; // valid?
```

```
ptr3=&fval; // valid?
```

```
ptr1= 0xB8000000; // valid?
```

```
ptr1= (int *) 0xB8000000; // valid?
```

- Some limited arithmetic operations
 - integer values can be added to and subtracted from a pointer variable
- ```
ptr2=ptr1 + 5; ptr2=ptr1 - 5
```

# Pointer Operation

---

- value of one pointer variable can be subtracted from another pointer variable of **the same type**

`int noElem=ptr2 -ptr1;`

- Increment and decrement operation are valid.

`ptr++, ptr--, ++ptr, --ptr`

- Other arithmetic operation are not allowed

- **Relational operations** - two pointer variables of the same type can be compared for equality, and so on

`ptr1 <= ptr2; ptr1==ptr2`

The most common is checking a pointer for NULL

`if(ptr == NULL)` equivalent to `if(!ptr)`

`if(ptr != NULL)` equivalent to `if(ptr)`

# Const Modifiers and Pointers

---

The use of the *const* modifier with pointers is confusing as it can mean two things

- `const int* cptrInt; // cptrInt is a pointer to a const int`

You can not change the value of the integer that `cptrInt` points to but you can change the pointer itself

- `int* const ptrcInt; // ptrcInt is a constant pointer to int`

You can change the value of the integer that `ptrcInt` points to but you can not change the pointer itself

Example:

```
double x=6.8, y=8.6;
```

```
const double *ptr1=&x;
```

```
double* const ptr2=&x;
```

```
ptr2=ptr1; // Is it valid?
```

```
*ptr1=8.7; // Is it valid?
```

# Pointer and Array

---

There is a close association between pointers and arrays

**The name of an array** is also a **constant pointer** to the first element of the array. Therefore an array element can be accessed using the dereference operator. Thus

`arr[3]` is equivalent to `*(arr + 3)`

**The offset operator([])** that we used to access the elements of an array is another dereference operator. They dereference the variable they follow just as `*` does, but they also add the number(offset) between brackets to the address being dereferenced.

Thus if a pointer `p` points to the first element of the array `arr`, it is possible to access the elements of the array through the pointer using the dereference operator or the offset operator

`*(p + 3)` is equivalent to `p[3]`// access the fourth element

# *Pointer and...*

---

```
int array[5] = { 23, 5, 12, 34, 17 }; // array of 5 ints
for (int i=0; i< 5; i++)
 cout << array[i] << endl; //using offset operator to access elements
for (int i=0; i< 5; i++)
 cout << *(array+i) << endl; //using pointer version to access elements
int *arrPtr=array;
for (int i=0; i< 5; i++)
 cout << arrPtr[i] << endl; using pointer to access elements
```

**Note:** Although an array name is a pointer, it can not be assigned because it is a constant pointer.

array=arrPtr; //Invalid even if both array and arrPtr point to the same type

# *Example*

---

Write a program that reverses the elements of one dimensional array without using another array. Use pointer concept to traverse the array.

# *Example*

---

Write a program that an array elements are palindrome. Use pointer concept to traverse the array.

# *Void Pointer*

---

The void type of pointer is a special type of pointer. In C++, void represents the absence of type, so void pointers are pointers that point to a value that has no type **(and thus also an undetermined length and undetermined dereference properties)**.

The **void pointer, also known as the generic pointer, is a special** type of pointer that can be pointed at objects of any data type. A void pointer is declared like a normal pointer, using the void keyword.

Syntax:

```
void * voidPtr;
```

Example:

```
int val1=6; float val2=3.14;
```

```
void * voidPtr=&val1;
```

```
voidPtr=&val2;
```

# *Void Pointer*

---

The programmer must note that void pointers cannot be dereferenced in the same manner as other pointers.

Direct dereferencing of void pointer is not permitted.

The programmer must change the pointer to void as any other pointer type that points to valid data types such as, int, char, float and then dereference it.

```
cout << *((float *) voidPtr); //the void pointer is casted to
//float pointer then dereferenced.
```

# *Void Pointer*

---

```
#include <iostream.h>
#include <conio.h>
int main()
{
 int inum[3] = {10,20,30},i;
 float fnum[4] = {1.2, 2.4, 3.7,9.8};
 void *pvoid = NULL;
 pvoid = inum;
 for (i=0; i<3; i++)
 cout<<*((int *)pvoid + i);
 // Same void pointer can be cast to float
 cout<<"\n";
 pvoid = fnum;
 for(i=0; i < 4; i++)
 cout<< *((float *) pvoid + i));
 getch();
}
```

# Pointer to Pointer

---

C++ allows the use of pointers that point to pointers, that these, in its turn, point to data (or even to other pointers). In order to do that, we only need to add an asterisk (\*) for each level of reference in their declarations:

Example:

```
char a; // a is type char
```

```
char * b; //b is type a pointer to char
```

```
char ** c; //c is type a pointer to pointer to char
```

```
char ***d; //d is type a pointer to pointer to pointer to char
```

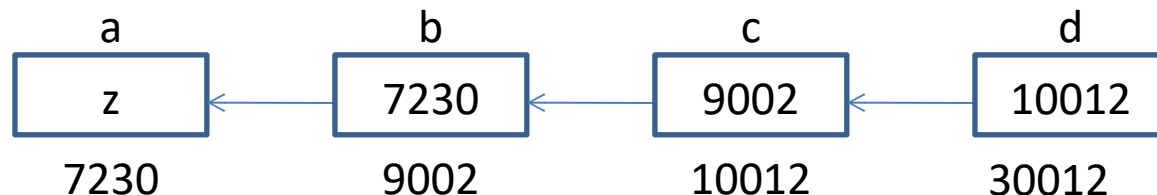
```
a = 'z';
```

```
b = &a;
```

```
c = &b;
```

```
d=&c;
```

supposing the randomly chosen memory locations for each variable are 7230, 8092 and 9002, it could be represented as:



# *Pointer to...*

---

```
#include <iostream.h>
int main()
{
 int num = 26;
 int *ptr;
 int **ptrToPtr;
 ptr = #
 ptrToPtr = &ptr;
 cout << " The value of num is: = " << num << "\n";
 cout << " The value of num is: = " << *ptr << "\n";
 cout << " The value of numis: = " << **ptrToPtr << "\n";
 return 0;
}
```